Technical Report: Book Metadata and Recommendation Analyt

Module: XJCO3011 Web Services and Web Data
Assessment: Coursework 1 (Individual API Development Project)
Repository: https://github.com/Yuk11119/Web_Services_cw1
API documentation: docs/api_documentation.pdf
Presentation link: TBD_SLIDES_LINK (to be replaced in Step 3)

1. Project Overview

This project delivers a data-driven RESTful API for book metadata, authors, reviews, and analytics. It satisfies required CRUD behavior, SQL-backed persistence, endpoint documentation, and demonstration readiness.

2. Stack Justification

- FastAPI for rapid endpoint delivery, validation, and OpenAPI generation.
- SQLAlchemy + relational schema constraints for structured data integrity.
- Alembic for versioned, reproducible migration workflow.
- JWT authentication for protected resources and standard API security.
- Pytest integration tests for evidence-backed functional verification.

3. Architecture and Design Decisions

The backend uses router/service/repository/model layering to separate HTTP contracts, business logic, and persistence.
Core entities are users, authors, books, and reviews, with unique ISBN, FK constraints, and rating bounds.

Architecture and Data Model Visuals

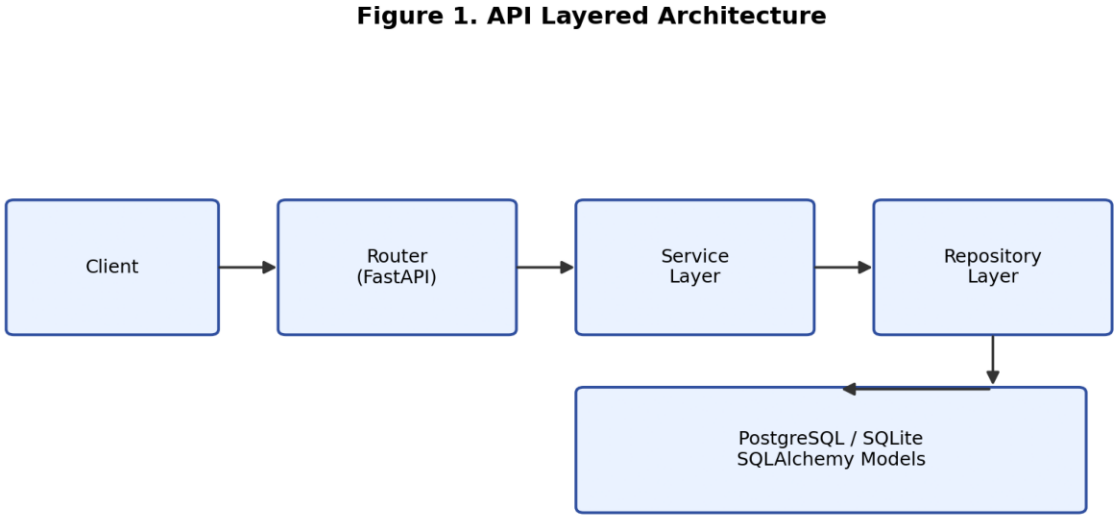

Figure 1. Layered API architecture (router -> service -> repository -> database).

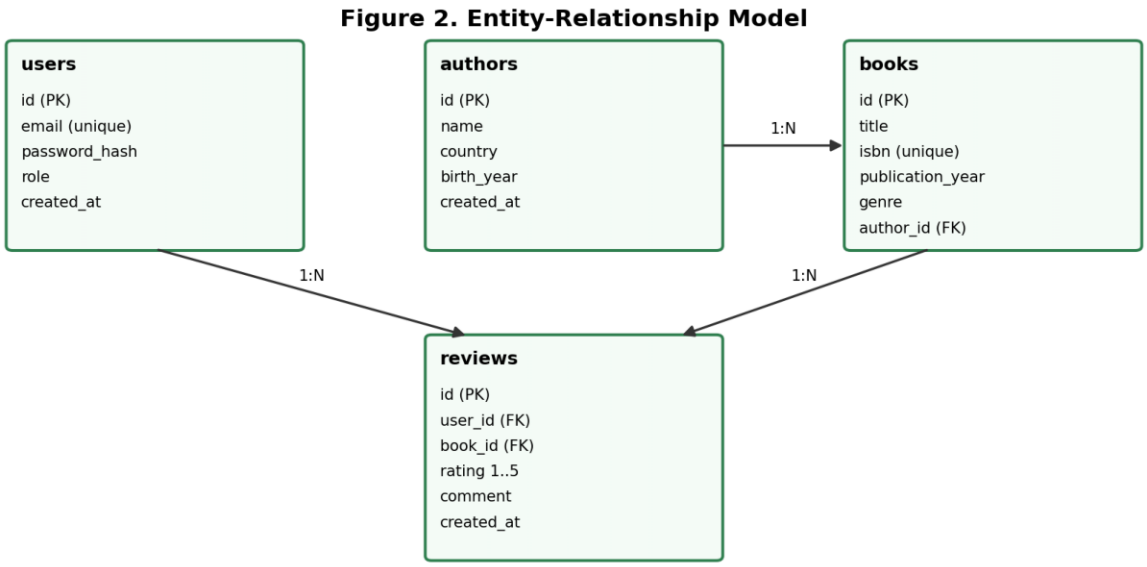

Figure 2. Entity-relationship model for users/authors/books/reviews.

Implementation and Validation

4. Implementation Highlights

- Authentication: /auth/register, /auth/login
- Authors CRUD: /authors
- Books CRUD + filters (genre/year/author_id): /books
- Reviews CRUD + filters (book_id/user_id/rating): /reviews
- Analytics: /analytics/genre-trends, /analytics/rating-distribution, /analytics/recommendations/{user_id}

Responses follow consistent envelopes:
- success: { data: ..., meta: ... }
- error: { error_code: ..., message: ..., details: ... }

5. Testing Strategy and Evidence

Integration-focused testing was implemented for auth, protected access, CRUD flow, conflict/validation, and analytics behavior.
Latest execution evidence:
- pytest -q -> 6 passed

6. Challenges and Lessons Learned

- Uniform error-contract handling across multiple routers.
- SQL compatibility for aggregation/recommendation queries.
- Maintaining modular architecture while preserving practical testability.

Lessons: strict layering and early integration tests reduced downstream debugging and contract drift.

Runtime Flow and Test Evidence Visuals

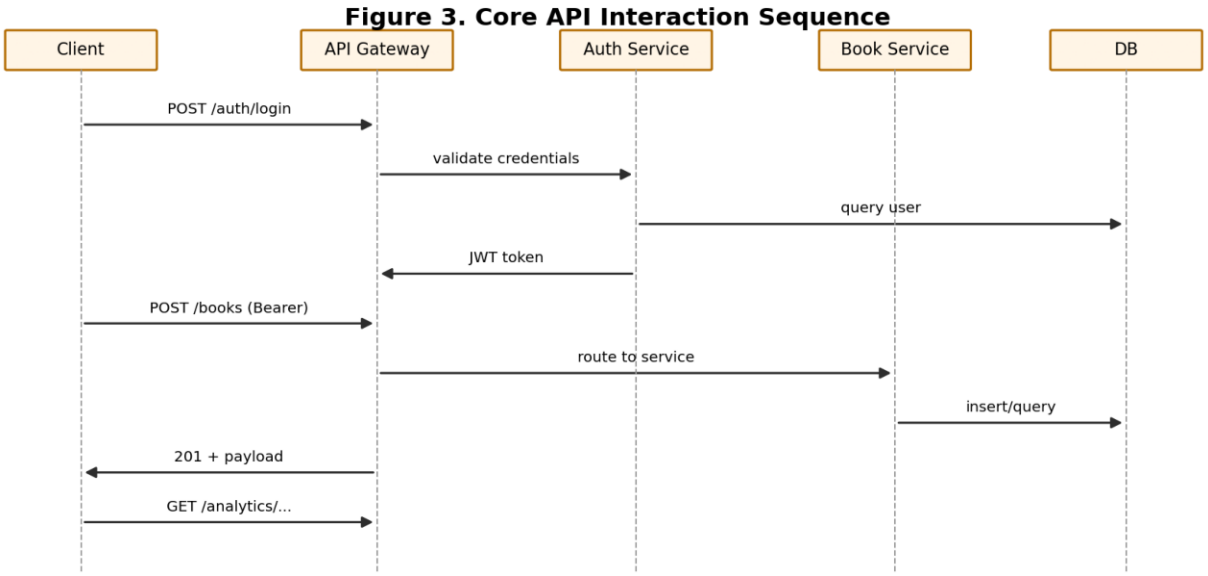

Figure 3. Core request sequence from login to protected CRUD and analytics.

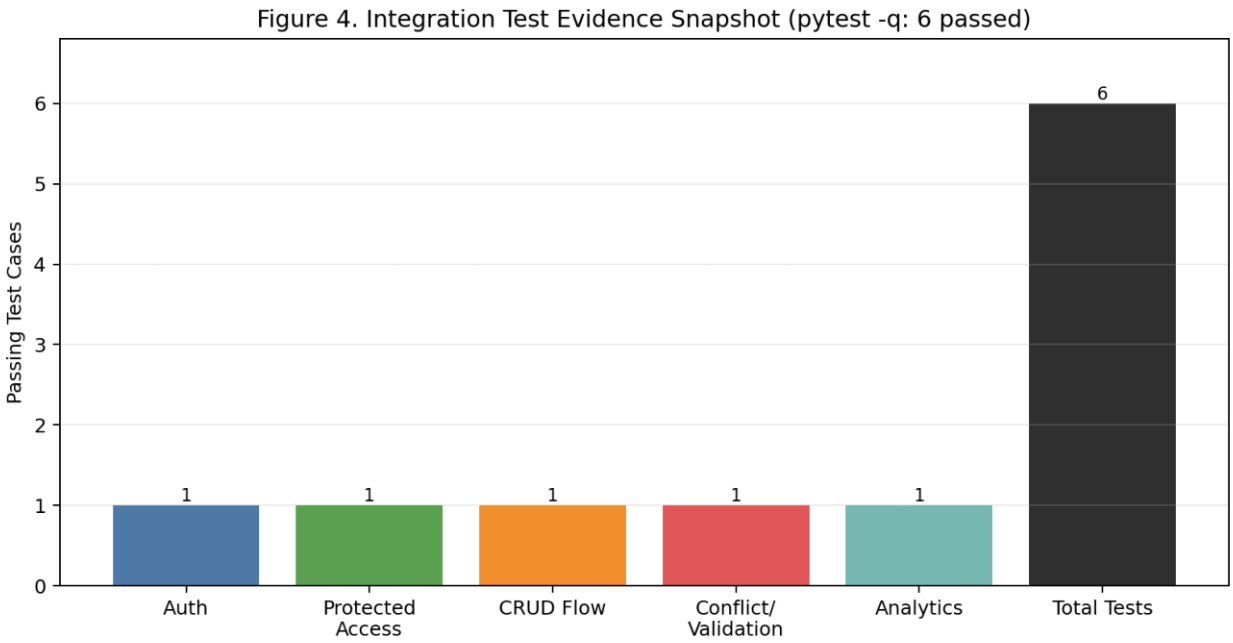

Figure 4. Integration evidence snapshot (pytest: 6 passed).

Limitations, Licensing, and GenAI Declaration

7. Limitations and Future Work

- Recommendation logic is heuristic and can be extended with collaborative/personalized methods.
- External dataset ingestion pipeline is not yet productionized.
- Advanced production controls (rate limiting, audit logging, token rotation) remain future work.

8. Dataset and Licensing

Current implementation uses small seeded demonstration data (scripts/seed_data.py).
Planned extension datasets: Open Library, Google Books metadata, or Kaggle sources subject to license checks.
Final submission must document exact dataset source URLs and terms.

9. GenAI Declaration and Reflection

Declared GenAI use included planning support, debugging assistance, and documentation drafting. Human verification was applied through manual review, local execution, and test confirmation before acceptance.

Integrity statement:
- AI outputs were treated as proposals, not authoritative final answers.
- Final engineering decisions and submission responsibility remained with the student.
- Report claims were restricted to implemented behavior in the repository.

Supporting appendix and logs:
- docs/genai_declaration_appendix.md
- docs/genai_logs/ (including test evidence text file)